Arrays



Soyeb Ghachi

Technical Trainer | Mentor |SME

Overview



- Introduction to Arrays
- Getting Array Size
- Multidimensional Arrays
- Rectangular Array
- Jagged Array
- System.Array Class

Introduction to Arrays in C#.Net

- Arrays allow you to store many values into one variable of the same type.
- It is not efficient to declare a separate variable for each value.
- Arrays in C# are reference types and are very much objects in their own right.

The following code shows how to declare references to an int array.

Syntax:

```
dataType[] arrayName = new dataType[size];
```

```
dataType[] arrayName = { value1, value2,
value3, ... };
```

Example:

```
Int[] odd=new int[]{3,5,7,9};
Int[] odd ={3,5,7,9};  //which can be shortened to
```

***Declare an array:**

```
string[] x = {"I", "am", "learning", "C#");
```

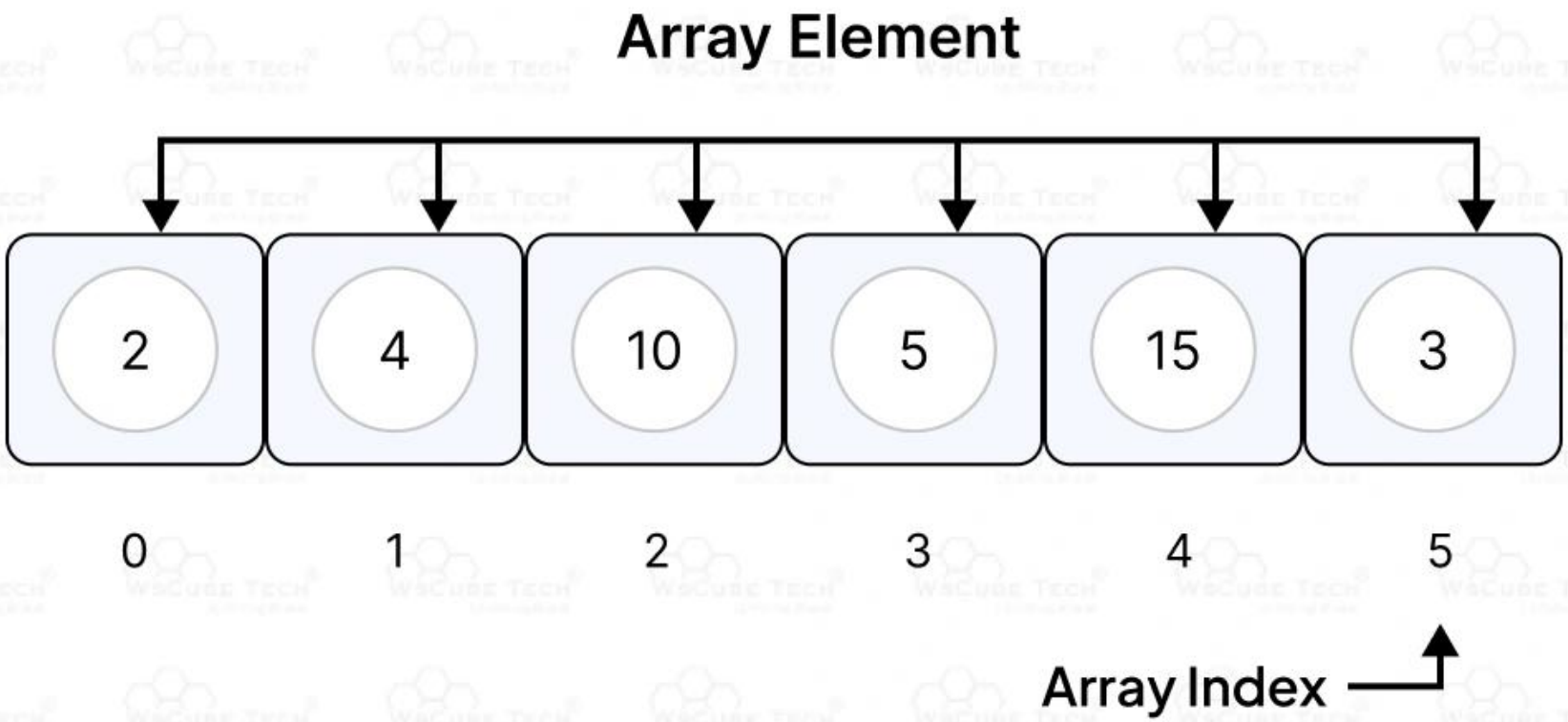
Index 0: I

Index 1: am

Index 2: learning

Index 3: C#

Array Data Structure



Getting Array Size

Length:

Knowing the size of your array is important in looping through each element in an array. The loop must know when to stop. The demonstration below gives an example of how to get the size or obtain the amount of elements in an array.

```
int[] numbers = { 1, 2, 3, 4, 5 };

Console.WriteLine(numbers.Length); // Output: 5
```

Multidimensional Arrays

- Multidimensional arrays are arrays within arrays also known as rectangular arrays. It may become necessary to nest an array in an array.

Syntax:

```
<Type>[,] ArrayName;

<Type>[,,,,,] <ArrayName>
```

Example:

```
int[,] matrix = new int[3, 4] {
    { 1, 2, 3, 4 },
    { 5, 6, 7, 8 },
    { 9, 10, 11, 12 }
};
```

Rectangular Array

Rectangular Array:

- A rectangular array, also known as a multidimensional rectangular array, is a type of array in C# where each row has the same number of elements.
- Rectangular arrays in C# have a fixed size, and the dimensions cannot be changed once the array is created.
- The number of rows and columns is determined at the time of initialization.

Syntax:

```
elementType[,] arrayName = new elementType[numRows, numColumns];
```

Example:

```
int[,] rectangularArray = new int[3, 4];
```

```
int[,] odd;
odd = new int[,] { { 3, 5, 7, 9 }, { 11, 13, 15, 17 } };
```

Jagged Arrays

C# Jagged Arrays:

- They are more flexible than the rectangular arrays. With Jagged arrays the rows don't have to be the same size.
- You can access and modify elements in a jagged array using the row and column indices just like a rectangular array.
- However, since each "row" is an independent array, the length and structure of each subarray can vary.

Syntax:

```
elementType[][] arrayName = new elementType[numRows][];
```

Example:

Declare Jagged Array

```
string[][] Families = new string [2][];
```

Initialize Jagged Array

```
 string[][] Families = new string[2][] {{"Jamal", "Jane", "Mitch", "Anthony", "Corel"}, {"Don",
"Joan"}};
```

Multi Dim. Jagged Arrays:

A multi-dimensional jagged array is a jagged array where each element is a multi-dimensional array.

```
int[][,] jaggedMultiDimArray = new int[2][,];


// First element: 2x2 matrix
jaggedMultiDimArray[0] = new int[,] { {1, 2}, {3, 4} };


// Second element: 3x1 matrix
jaggedMultiDimArray[1] = new int[,] { {5}, {6}, {7} };


Console.WriteLine(jaggedMultiDimArray[0][1, 1]);  // Output: 4
Console.WriteLine(jaggedMultiDimArray[1][2, 0]);  // Output: 7
```

System.Array Class

The System.Array type is the abstract base type of all array types.

Following are the methods of System.Array:

Non-Static Members

- CopyTo()
- GetValue()
- SetValue()
- GetUpperBound()
- GetLowerBound()
- GetLength()
- GetType()

Static Members

- Copy()
- Reverse()
- Sort()
- Clear()
- IndexOf()